

Mythos Fix — Autonomous Vulnerability Remediation Platform (V1 → V2)

1. Overview

Mythos Fix is a **governed autonomous remediation control plane** designed to:

- Ingest and normalize vulnerabilities across heterogeneous infrastructure
- Compute risk scoring deterministically
- Use LLMs only for structured remediation planning
- Enforce policy constraints via OPA
- Require Human-in-the-Loop (HITL) approval for execution
- Execute patching via agentless mechanisms (V1)
- Evolve into distributed agent-based execution (V2)

It is NOT an autonomous execution system. It is a **controlled decision + orchestration system**.

2. Core Principles

2.1 Safety First Architecture

- LLMs NEVER execute actions
- All actions are structured JSON outputs
- Policy engine is authoritative gatekeeper
- Human approval is mandatory for all remediation

2.2 Deterministic Execution

- Execution layer is deterministic and idempotent
- No free-form shell execution allowed
- All commands are templated or adapter-based

2.3 Hybrid Autonomy

- Phase 1: Fully agentless
- Phase 2: Hybrid agent + agentless mesh

2.4 Auditability

- Every action is logged immutably
 - Full traceability from CVE → execution → outcome
-

3. System Architecture

3.1 High-Level Architecture

- Control Plane:
 - FastAPI (API Gateway)
 - LLM Planner Engine
 - Policy Engine (OPA)
 - Approval Engine (HITL)
 - Orchestrator (Temporal)
 - PostgreSQL (state)
 - Data Plane:
 - SSH / WinRM / Kubernetes API / AWS SSM
 - Ansible execution layer
 - Observability:
 - Prometheus
 - Grafana
 - OpenTelemetry
 - Loki
-

4. Technology Stack

Backend

- Python 3.11+
- FastAPI
- Pydantic v2
- SQLAlchemy
- Alembic

Orchestration

- Temporal.io (primary)
- Celery (fallback / MVP only)

Execution Layer (V1)

- Ansible Runner (Python)
- Paramiko (SSH)
- Kubernetes Python Client

- AWS Boto3 (SSM)

AI Layer

- OpenAI / Claude API (primary)
- Ollama (fallback local inference)

Policy Engine

- Open Policy Agent (OPA)

Data Layer

- PostgreSQL (system of record)
- Redis (queues/cache)
- Kafka (event streaming)

Frontend

- Next.js dashboard
- Microsoft Teams Bot (Adaptive Cards)

5. Core Data Model

5.1 Assets

```
asset_id UUID PRIMARY KEY
hostname TEXT
ip TEXT
os_type TEXT
environment TEXT
criticality_score INT
```

5.2 Vulnerabilities

```
cve_id TEXT
asset_id UUID
package TEXT
installed_version TEXT
fixed_version TEXT
cvss_score FLOAT
exploitability_score FLOAT
risk_score FLOAT
```

5.3 Remediation Plans

```
plan_id UUID
asset_id UUID
generated_by_llm BOOLEAN
status TEXT
created_at TIMESTAMP
```

5.4 Patch Jobs

```
job_id UUID
plan_id UUID
execution_type TEXT
status TEXT
rollback_available BOOLEAN
```

5.5 Approvals

```
approval_id UUID
plan_id UUID
approver TEXT
decision TEXT
timestamp TIMESTAMP
```

5.6 Audit Log (Immutable)

```
event_id UUID
actor TEXT
action TEXT
payload JSONB
timestamp TIMESTAMP
```

6. Vulnerability Processing Pipeline

Step 1: Ingestion

Sources:

- NVD CVE feeds
 - OSV.dev
 - Trivy scans
 - Cloud security APIs
-

Step 2: Normalization

All vulnerabilities normalized into:

```
{
  "asset_id": "...",
  "cve": "...",
  "package": "...",
  "installed_version": "...",
  "fixed_version": "...",
  "severity": "HIGH",
  "exploitability": 0.0-1.0
}
```

Step 3: Risk Scoring Engine

Risk score computed deterministically:

Risk Score =

- CVSS score
 - Exploit availability
 - Asset criticality
 - Exposure level
 - Environment sensitivity
-

7. LLM Remediation Engine

7.1 Purpose

Convert vulnerability context into structured remediation plans.

7.2 Hard Constraints

- Must output valid JSON schema only
- Must not execute commands
- Must not decide final approval

7.3 Output Schema

```
{
  "actions": [
    {
      "type": "patch",
      "target": "openssl",
      "method": "apt",
      "version": "latest",
      "restart_required": false,
      "rollback": "previous version reinstall"
    }
  ],
  "risk": "medium",
  "explanation": "..."
}
```

8. Policy Engine (OPA)

Responsibilities

- Validate remediation plans
- Enforce environment rules
- Enforce safety constraints

Example Rules

- No production patching during business hours
 - No kernel updates without approval escalation
 - No restart-required actions auto-approved
-

9. Human-in-the-Loop (HITL)

Approval Channels

- Web Dashboard
- Microsoft Teams Bot

Workflow

1. LLM generates plan
 2. Policy engine validates
 3. Approval request created
 4. Analyst approves/rejects/modifies
 5. Temporal triggers execution
-

10. Execution Layer (Agentless V1)

Execution Adapters

- SSH Adapter (Linux systems)
 - WinRM Adapter (Windows systems)
 - Ansible Adapter (bulk execution)
 - Kubernetes Adapter (cluster workloads)
 - AWS SSM Adapter (cloud instances)
-

Execution Safety Rules

- Always run dry-run first
 - Always validate policy before execution
 - Always require approval token
 - Always support rollback plan
-

11. Temporal Orchestration Workflow

Workflow Steps

1. Receive remediation plan
2. Validate policy (OPA)
3. Request approval (HITL)
4. Wait for approval event
5. Execute patch job
6. Verify outcome
7. Update system state

8. Emit audit event

12. Dashboard System

Modules

- Global Risk Dashboard
 - CVE Explorer
 - Patch Execution Queue
 - Approval Center (Teams synced)
 - Audit Timeline Viewer
-

13. Microsoft Teams Integration

Features

- Adaptive Cards for approvals
 - Inline approve/reject buttons
 - Contextual CVE summaries
 - Asset impact visualization
-

14. Security Model

Identity & Access

- RBAC (Admin / Analyst / System)
- Service accounts for execution

Secrets Management

- Hashicorp Vault integration

Communication Security

- TLS everywhere
- mTLS for future agent system

Auditability

- Immutable logs
 - Full execution trace
-

15. Scaling Design (10,000+ Nodes)

Requirements

- Kafka event streaming backbone
 - Horizontal Temporal workers
 - Batched execution (50–200 nodes per batch)
 - Adaptive throttling per environment
-

16. Failure Handling

- Idempotent execution required
 - Retry-safe workflows
 - Partial failure recovery supported
 - Rollback strategy mandatory per patch plan
-

17. V2 Agent System (Future Expansion)

Agent Responsibilities

- Receive signed execution tasks
- Execute locally
- Report telemetry
- Enforce local policy cache

Agent Stack

- Python asyncio daemon
 - gRPC communication
 - mTLS authentication
 - Signed command verification
-

18. Deployment Model

V1 Deployment

- Dockerized FastAPI service
- PostgreSQL + Redis
- Temporal server
- Ansible execution workers

V2 Deployment

- Agent fleet rollout
 - Central control plane
 - Hybrid execution model
-

19. Key Risks

- LLM hallucination (mitigated via schema enforcement)
 - Patch side effects (mitigated via rollback + HITL)
 - Credential leakage (mitigated via Vault)
 - Execution inconsistency (mitigated via idempotent workflows)
-

20. Build Roadmap

Phase 1 (MVP)

- FastAPI backend
- CVE ingestion
- LLM planner
- SSH/Ansible execution
- Manual approvals
- Basic dashboard

Phase 2

- Temporal workflows
- OPA policy engine
- Teams integration
- audit system

Phase 3

- Kafka scaling backbone
- 10k node support
- multi-environment orchestration

Phase 4

- Agent system rollout
 - hybrid execution mesh
 - enterprise compliance features
-

21. Final System Philosophy

Mythos Fix is a:

Controlled autonomous remediation operating system

Not an LLM tool.

The LLM is:

- a planner
- a translator
- a suggestion engine

NOT:

- an executor
 - a decision authority
-